

Course Code	Course Title	L	R	P	Cr
	AI-Assisted Problem Solving Using Python Programming				
Program Core					

Day 5 Session:
Functions, Recursion,
Lambda & Modules

Day 5

- Function Parameters
- Scope
- Recursion
- Lambda Functions
- Modules & Libraries

Topics

- Function Parameters
- Variable Scope
- Recursion
- Lambda Functions
- Modules & Libraries

Curiosity Question

Think Before We Start...

"How do applications like Google Search, Instagram, Amazon, and ChatGPT organize millions of lines of code without becoming impossible to manage?"

Answer: Using Function Concepts

Types of Functions

1. No Arguments – No Return

```
def hello():  
    print("Hello")
```

2. Arguments – No Return

```
def display(name):  
    print(name)
```

3. No Arguments – Return Value

```
def pi():  
    return 3.14
```

4. Arguments – Return Value

```
def multiply(a,b):  
    return a*b
```

Scope of Variables: Only inside function

Local Variable

```
def demo():
```

```
    x=20
```

```
    print(x)
```

Global Variable: Accessible everywhere

```
x=100
```

```
def demo():
```

```
    print(x)
```

Local vs Global

Local

Created inside function

Temporary

Cannot access outside

Global

Outside function

Entire program

Accessible everywhere

Recursion

- Python also accepts function recursion, which means a defined function can call itself.
- Recursion is a common mathematical and programming concept. It means that a function calls itself. This has the benefit of meaning that you can loop through data to reach a result

What is Recursion?

A function calling itself until a stopping condition occurs.

Real-life examples

- Mirror reflection
- Russian Dolls
- Folder inside Folder
- Family Tree

Recursion : Example

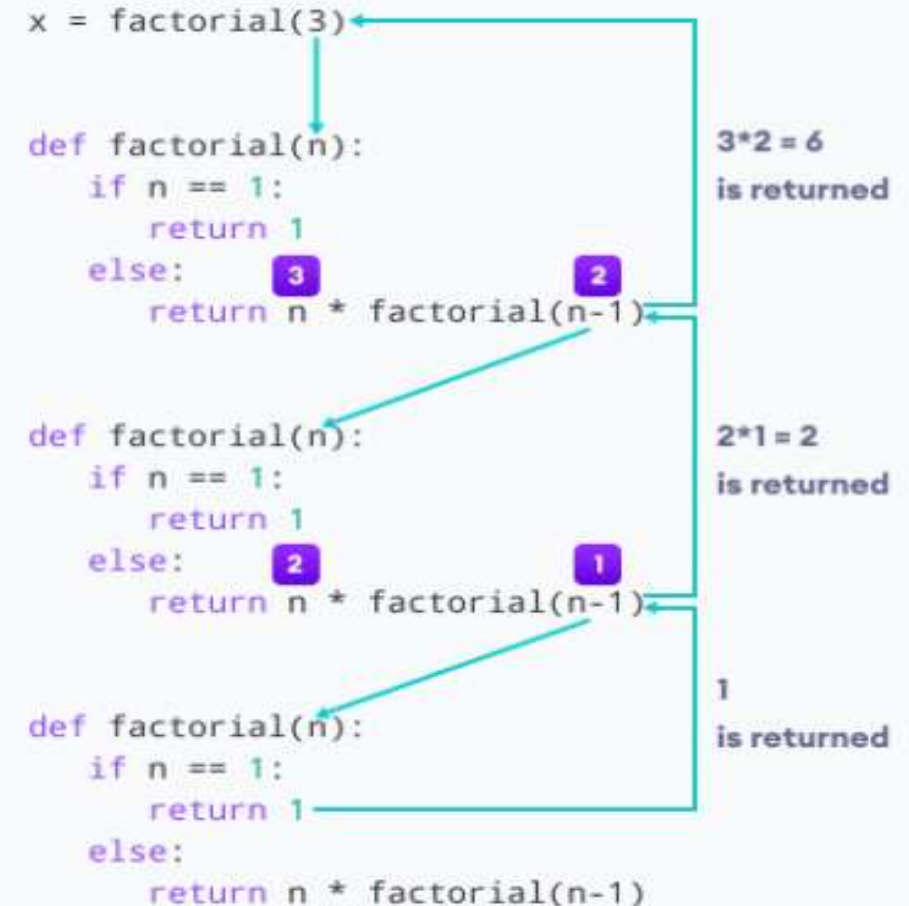
```
def factorial(x):
    """This is a recursive function
    to find the factorial of an integer"""

    if x == 1:
        return 1
    else:
        return (x * factorial(x-1))

num = 5
print("The factorial of", num, "is", factorial(num))

===== RESTART: C:/python programs/factorial.py =====
The factorial of 5 is 120
```

```
factorial(3)    # 1st call with 3
3 * factorial(2) # 2nd call with 2
3 * 2 * factorial(1) # 3rd call with 1
3 * 2 * 1      # return from 3rd call as number=1
3 * 2          # return from 2nd call
6              # return from 1st call
```



Working of a recursive factorial function

Python Lambda

- A lambda function is a small anonymous function.
- A lambda function can take any number of arguments, but can only have one expression.

Syntax

lambda arguments : expression

The expression is executed and the result is returned

Example

Add 10 to argument a, and return the result

```
x = lambda a: a + 10
print(x(5))
```

15

```
x = lambda a, b: a * b
print(x(5, 6))
```

30

```
x = lambda a, b, c: a + b + c
print(x(5, 6, 2))
```

13

Lambda with map()**Example:**

```
numbers=[1,2,3,4,5]
```

```
result=list(map(lambda x:x*x,numbers))
```

```
print(result)
```

Output

```
[1,4,9,16,25]
```

Lambda with filter()

```
numbers=[10,15,20,25,30]
```

```
result=list(filter(lambda x:x%2==0,numbers))
```

```
print(result)
```

Output

```
[10,20,30]
```


Python Modules

Why Modules?

Instead of writing everything ourselves

Types of modules:

- Built-in Modules
- User-defined Modules

Python provides Built-in Modules

- math
- random
- statistics
- datetime
- os
- numpy
- pandas

The Math Module

Python has also a built-in module called `math`, which extends the list of mathematical functions.

To use it, you must **import** the `math` module

Syntax: **import math**

When you have imported the `math` module, you can start using methods and constants of the module.

Example:

The `math.sqrt()` method for example, returns the square root of a number

The `math.ceil()` method rounds a number upwards to its nearest integer, and the `math.floor()` method rounds a number downwards to its nearest integer, and returns the result

The `math.pi` constant, returns the value of PI (3.14...)

```
import math
x = math.sqrt(64)
print(x)
```

```
k = math.ceil(1.4)
l = math.floor(1.4)
```

```
print(k) # returns 2
print(l) # returns 1
```

```
m = math.pi
print(m)
```

```
===== RESTART: D:/mathmodule.py
```

```
8.0
```

```
2
```

```
1
```

```
3.141592653589793
```

Method	Description	Example	Output
<code>math.sqrt(x)</code>	Square root	<code>math.sqrt(25)</code>	5.0
<code>math.pow(x, y)</code>	Power	<code>math.pow(2, 3)</code>	8.0
<code>math.ceil(x)</code>	Round up	<code>math.ceil(4.2)</code>	5
<code>math.floor(x)</code>	Round down	<code>math.floor(4.9)</code>	4
<code>math.fabs(x)</code>	Absolute value	<code>math.fabs(-12.5)</code>	12.5
<code>math.factorial(x)</code>	Factorial	<code>math.factorial(5)</code>	120
<code>math.gcd(a, b)</code>	Greatest Common Divisor	<code>math.gcd(24, 36)</code>	12
<code>math.lcm(a, b)</code>	Least Common Multiple	<code>math.lcm(12, 18)</code>	36
<code>math.exp(x)</code>	e raised to x	<code>math.exp(2)</code>	7.389...
<code>math.log(x)</code>	Natural logarithm	<code>math.log(10)</code>	2.302...
<code>math.log10(x)</code>	Base-10 logarithm	<code>math.log10(100)</code>	2.0
<code>math.sin(x)</code>	Sine (radians)	<code>math.sin(math.pi/2)</code>	1.0
<code>math.cos(x)</code>	Cosine (radians)	<code>math.cos(0)</code>	1.0
<code>math.tan(x)</code>	Tangent (radians)	<code>math.tan(math.pi/4)</code>	1.0
<code>math.degrees(x)</code>	Radians to degrees	<code>math.degrees(math.pi)</code>	180.0
<code>math.radians(x)</code>	Degrees to radians	<code>math.radians(180)</code>	3.14159
<code>math.dist(p, q)</code>	Distance between two points	<code>math.dist((0,0),(3,4))</code>	5.0
<code>math.hypot(x, y)</code>	Euclidean distance	<code>math.hypot(3,4)</code>	5.0
<code>math.isqrt(x)</code>	Integer square root	<code>math.isqrt(27)</code>	5
<code>math.prod(iterable)</code>	Product of elements	<code>math.prod([2,3,4])</code>	24

Random Module

Python **Random module** :

- Random Module is an in-built module of Python which is used to generate random numbers.
- These are pseudo-random numbers means these are not truly random.
- This module can be used to perform random actions such as generating random numbers, print random a value for a list or string, etc.

Example: Printing a random value from a list

```
import random
# prints a random value from the list
list1 = [1, 2, 3, 4, 5, 6]
print(random.choice(list1))
```

Output: 2

Method	Description	Example	Sample Output
<code>random.random()</code>	Random float between 0.0 and 1.0	<code>random.random()</code>	0.7465
<code>random.randint(a, b)</code>	Random integer between a and b (inclusive)	<code>random.randint(1,10)</code>	7
<code>random.randrange(start, stop, step)</code>	Random number from a range	<code>random.randrange(1,20,2)</code>	15
<code>random.uniform(a, b)</code>	Random float between a and b	<code>random.uniform(1,10)</code>	5.82
<code>random.choice(sequence)</code>	Random element from a sequence	<code>random.choice(["A","B","C"])</code>	"B"
<code>random.choices(sequence, k=n)</code>	Select multiple elements (with replacement)	<code>random.choices(["A","B","C"], k=3)</code>	['B','A','C']
<code>random.sample(sequence, k)</code>	Select unique elements	<code>random.sample([1,2,3,4,5], 3)</code>	[2,5,1]
<code>random.shuffle(list)</code>	Shuffle a list	<code>random.shuffle(numbers)</code>	Random order
<code>random.seed(x)</code>	Set seed for reproducible results	<code>random.seed(10)</code>	Same output every run
<code>random.getrandbits(n)</code>	Random integer with n random bits	<code>random.getrandbits(8)</code>	173

Module

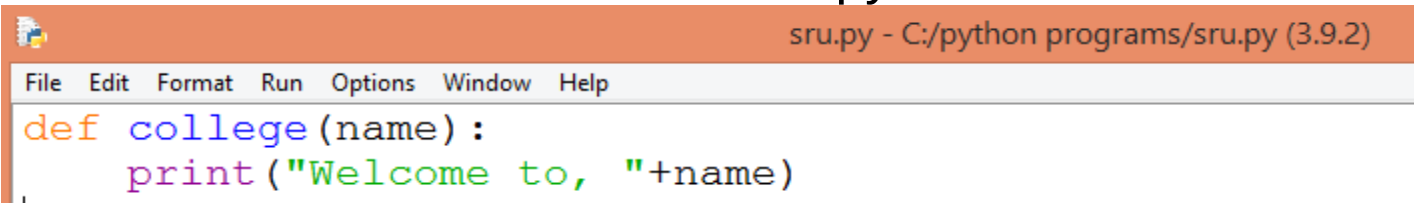
- Module is a file containing a set of functions you want to include in your application.
- Consider a module to be the same as a code library

Create a Module

To create a module just save the code you want in a file with the file extension .py

Example

Save this code in a file named sru.py



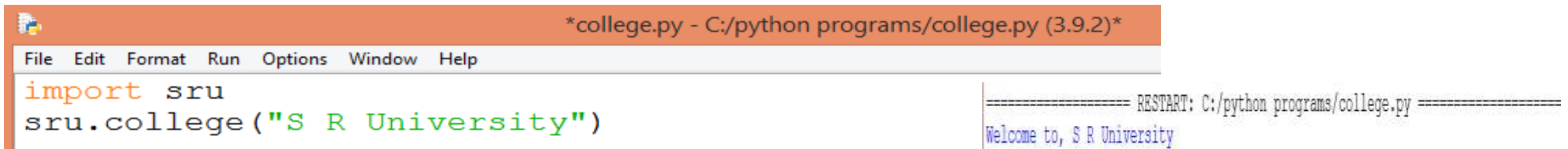
```
sru.py - C:/python programs/sru.py (3.9.2)
File Edit Format Run Options Window Help
def college(name):
    print('Welcome to, '+name)
```

Use a Module

Now we can use the module we just created, by using the **import** statement:

Example

Import the module named **sru**, and call the **college** function:



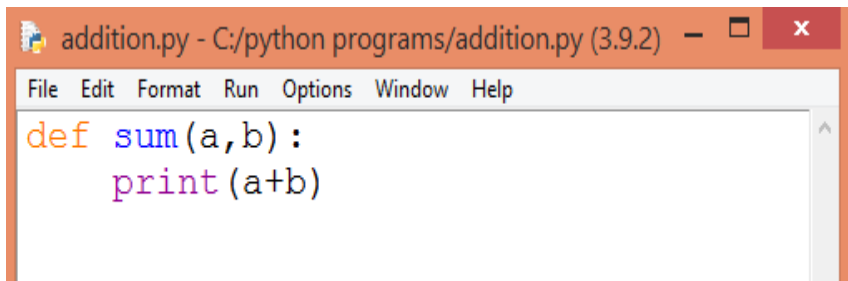
```
*college.py - C:/python programs/college.py (3.9.2)*
File Edit Format Run Options Window Help
import sru
sru.college('S R University')
===== RESTART: C:/python programs/college.py =====
Welcome to, S R University
```

Note: When using a function from a module, use the syntax: **module_name.function_name**.

Create a Module

To create a module just save the code you want in a file with the file extension .py

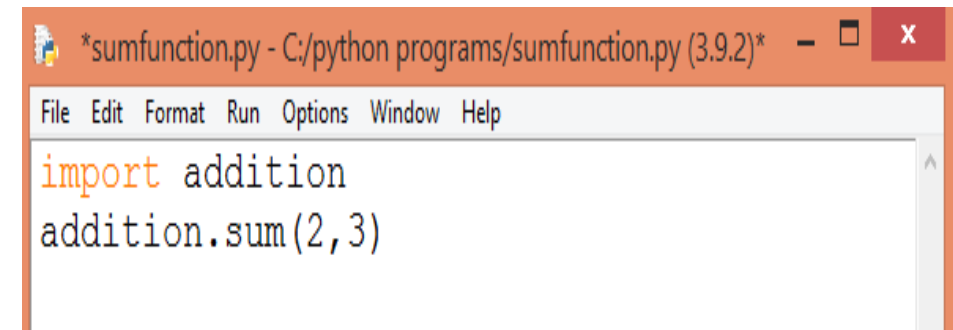
Save as **addition.py**



```
def sum(a,b):
    print(a+b)
```

Use a Module

- Now we can use the module we just created, by using the **import** statement:
- Import the module named **addition**, and call the **sum** function:



```
import addition
addition.sum(2,3)
```

Output:

Run **sumfunction.py** file

```
===== RESTART: C:/python programs/sumfunction.py =====
```

Variables in Module

The module can contain functions, as already described, but also variables of all types (arrays, dictionaries, objects etc)

Example:

```
student1 = {
    "name": "John",
    "age": 19,
    "branch": "CSE&AIML",
    "college": "S R University",
    "country": "India"
}

student2 = {
    "name": "vijay kumar",
    "age": 20,
    "branch": "ECE",
    "clg": "SRIIT",
    "country": "India"
}
```

```
#Import the module named personnaldetails, and access the student1 dictionary

import personnaldetails
name = personnaldetails.student1["name"], personnaldetails.student2["name"]
print(name)

age = personnaldetails.student1["age"]
print(age)

college = personnaldetails.student1["college"],personnaldetails.student2["clg"]
print(college)

country = personnaldetails.student1["country"]
print(country)
```

```
===== RESTART: C:/python programs/student.py =====
('John', 'vijay kumar')
19
('S R University', 'SRIIT')
India
```


Python Packages

- A **Python module** may contain several classes, functions, variables, etc.
- whereas a Python package can contains several module.
- In simpler terms a package is folder that contains various modules as files.

Creating Package: Create a folder with name **mypckg**

module1.py
module1.py - D:\

File Edit Format Run Options Window Help

```
def sru():
    print("Welcome to SR University")
```

module2.py

File Edit Format Run Options Window Help

```
def sum(a, b):
    return a+b
```

init.py

File Edit Format Run Options Window Help

```
from .module1 import sru
from .module2 import sum
```

The hierarchy of the our package looks like this –

Local Disk (D:) > APTT > mypckg				
Name	Date modified	Type	Size	
init.py	4/12/2022 12:14 PM	PY File	0 KB	
module1.py	4/12/2022 12:13 PM	PY File	1 KB	
module2.py	4/12/2022 12:15 PM	PY File	1 KB	

Create a file with name **example_package.py** beside of folder **mypckg**

```
from mypckg import module1
from mypckg import module2
```

```
module1.sru()
res = module2.sum(1, 2)
print(res)
```

```
===== Output: ===== RESTART: D:/APTT/example_package.py
Welcome to SR University
3
```



Thank you